Advanced Python

Day 3

Methods Inside Class (Object Behavior)

Goal: Make objects perform actions using methods

What is a Method?

A method is simply a function inside a class.

</> Normal Function

Functions defined outside of any class context.

```
def greet(): print("Hello")
```

Method Inside Class

Functions defined inside a class, belonging to objects.

```
class Person: def greet(self):
print("Hello")
```

Why Methods?

Objects need to hold information and do things with it.

Store Data

Objects hold properties. For a Bank Account, this means variables like `name` and `balance`.

Perform Actions

Objects execute operations. For a Bank Account, this means actions like `deposit()` and `withdraw()`.

The Formula

Objects =
Data + Behavior

Basic Method Example

Defining a method inside a `Student` class and calling it.

```
class Student: def __init__(self, name): self.name
= name def greet(self): print("Hello,", self.name)
# Creating an object and calling the method s1 =
Student("Saurav") s1.greet()
```

```
Output:
Hello, Saurav
```

"Methods must **always** include `self` as the first parameter."

Without self → Error.

How Methods Work Internally

Why do we need `self`? Because the object needs to know it's acting on its own data.

When you write:

You call the method directly on the object variable.

```
s1.greet()
```

Python actually runs:

The class method is called, and the object is automatically passed as the first argument (self).

```
Student.greet(s1)
```

Real Example: Bank Account

Let's model a Bank Account where the data (balance) is managed by behaviors (methods).

```
class BankAccount: def __init__(self, name, balance): self.name = name self.balance = balance def show_balance(self): print("Balance:", self.balance) # Usage acc = BankAccount("Saurav", 1000) acc.show_balance()
```

Adding a Deposit Method

The Code

```
class BankAccount: def __init__(self, balance): self.balance = balance def deposit(self, amount): self.balance += amount print("New Balance:", self.balance)
```

The Execution

The object's internal data changes when the method is called.

```
acc = BankAccount(1000) acc.deposit(500)
```

```
Output:
New Balance: 1500
```

Adding a Withdraw Method

The Code

```
def withdraw(self, amount): if amount ⩽
self.balance: self.balance -= amount
print("Withdraw successful") else:
print("Insufficient balance")
```

Logic Lives Inside Object

The method encapsulates the business logic. It checks the state (`self.balance`) before performing the action.

- Protects data from invalid updates.
- Keeps related logic bundled together.

Multiple Methods Together

One object can exhibit multiple behaviors.

```
class Dog: def __init__(self, name): self.name = name
def bark(self): print(self.name, "is barking") def
sleep(self): print(self.name, "is sleeping") d1 =
Dog("Tommy") d1.bark() d1.sleep()
```

🧪 Output Prediction

The Code

```
class Test: def __init__(self, x): self.x =
x def show(self): print(self.x) t1 =
Test(10) t2 = Test(20) t1.show() t2.show()
```

The Result

```
10
20
```

Key Concept:

Each object stores and manages a completely separate, independent value in memory.

Practice Tasks

Task 1: Car Class

Create a class named `Car`

- **Attributes:** `brand`, `speed`
- **Method:** `show_info()`

Task 2: Employee Class

Create a class named `Employee`

- **Attributes:** `name`, `salary`
- **Method:** `increase_salary(amount)`

More Practice Tasks

Task 3: Mobile Class

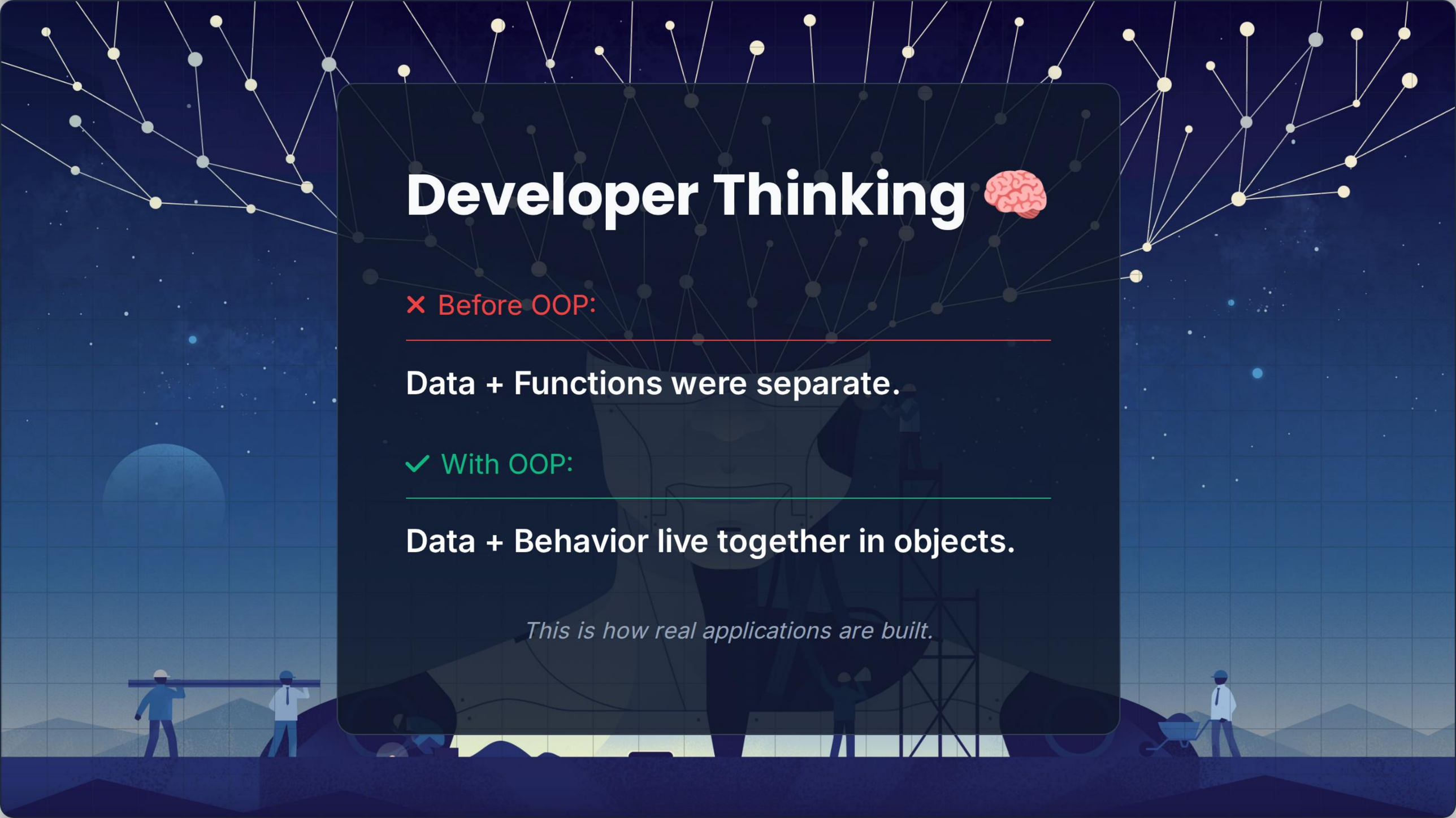
Create a class modeling a Mobile phone.

- **Attributes:** `brand`, `battery`
- **Method:** `charge()` (increases battery)

Task 4: Rectangle Class

Create a class for geometric calculations.

- **Attributes:** `length`, `width`
- **Method:** `area()` (returns length * width)


Developer Thinking 🧠
✕ Before OOP:
Data + Functions were separate.
✓ With OOP:
Data + Behavior live together in objects.
This is how real applications are built.

Day 3 Summary

- Learned what **methods** are and how they differ from normal functions.
- Understood **object behavior** (performing actions).
- Used the `self` parameter correctly to reference the object.
- **Modified object data** safely using methods (e.g., bank deposits).

You are now writing behavior-driven objects!

Honest Check 😄

Did you create your own classes and test methods or just read the examples?

If you completed the practice tasks, reply with:

🔥 **"Day 3 Advanced Done"**

Image Sources

https://sunscrapers.com/cdn-cgi/image/w=1600,h=818/https://sunscrapers-website-media-prod.s3.eu-central-1.amazonaws.com/images/Data-Visualization.width-1600.format-webp.webp

Source: sunscrapers.com

https://media.istockphoto.com/id/1987825543/vector/futuristic-digital-banking-interface-with-credit-cards-and-secure-transactions.jpg?s=612×612&w=0&k=20&c=OWwn4exb5IW6XjZ5bE6LRWGtoduVvyeLOrwmOcl7l-c=

Source: www.istockphoto.com

https://img.freepik.com/free-vector/cute-simple-dog-cartoon-character_1308-98028.jpg?semt=ais_hybrid&w=740&q=80

Source: www.freepik.com

https://www.quantamagazine.org/wp-content/uploads/2019/01/Neural_Networks_2880×1620.jpg

Source: www.quantamagazine.org